INTRODUCTION TO DEEP LEARNING

Course number: 00240332

Lecture 10: Word representation

Xiaolin Hu (胡晓林)

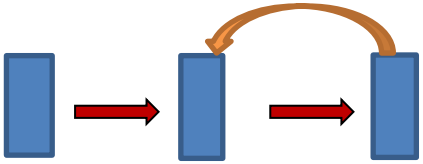
Dept. of Computer Science and Technology

Tsinghua University

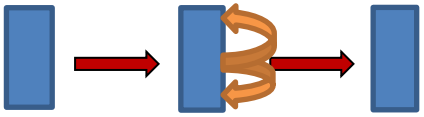
Last lecture review

I. Simple RNNs

Jordan network



Elman network



Bidirectional RNN

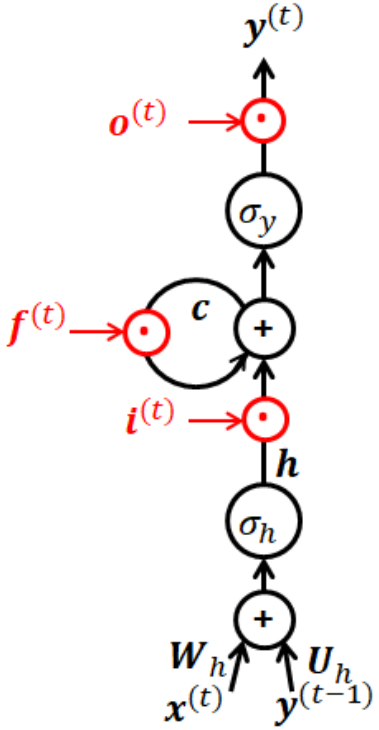
Deep RNN

BPTT

Teacher forcing

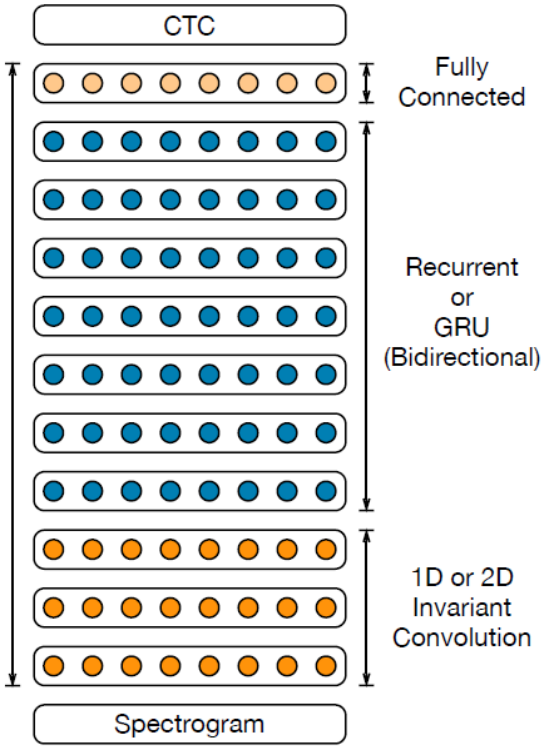
II. Gated RNNs

LSTM



III. Speech recog.

Deep Speech 2



Outline

I. Introduction to NLP
II. Word representation
III. Word2vec
IV. Summary

Natural language processing

- **Natural language processing** (**NLP**) is a field of computer science, artificial intelligence and computational linguistics concerned with the interactions between computers and human (natural) languages, and, in particular, concerned with programming computers to fruitfully process large natural language corpora.

- Involve natural language understanding, natural language generation (frequently from formal, machine-readable logical forms), connecting language and machine perception, managing human-computer dialog systems, or some combination thereof.

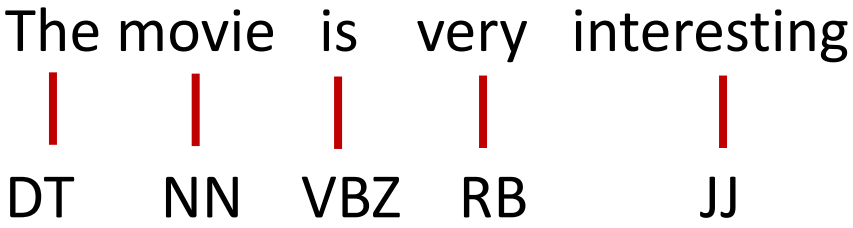
From https://en.wikipedia.org/wiki/Natural_language_processing

Typical tasks

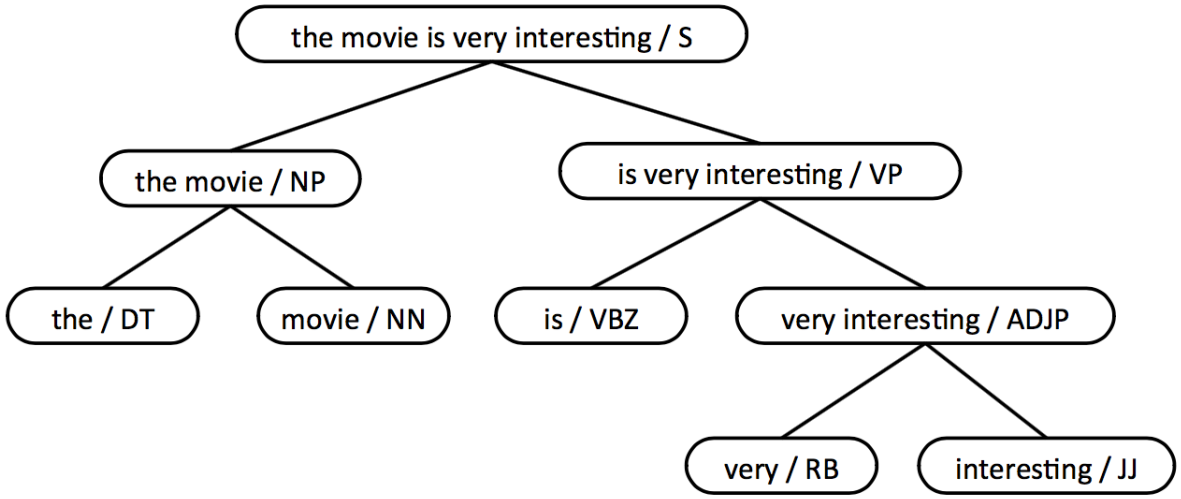
- Tagging and parsing
- Question answering and dialogue systems
- Text/document classification
- Sentiment analysis and opinion mining
- Machine translation
- Text generation
- Etc.

Tagging and parsing

Part-of-speech (POS) Tagging



Syntactic Parsing



Slides from Prof. Minlie Huang

Question answering and dialogue systems

Q: How many people live in Beijing?

A: *About **22 million**.*

Q: What creates a cloud

A: *in meteorology , a cloud is **a visible mass of liquid droplets** or frozen crystals made of water or various chemicals suspended in the atmosphere above the surface of a planetary body.*

x_0	*good morning*
y_0	*hello what can i help you with today*
x_1	*can you book a table in a cheap price range in london*
y_1	*i'm on it*
x_2	*<SILENCE>*
y_2	*any preference on a type of cuisine*
x_3	*with french food*
y_3	*how many people would be in your party*
x_4	*for four please*
y_4	*ok let me look into some options for you*
x_5	*<SILENCE>*
y_5	*api_call french london four cheap*

Slides from Prof. Minlie Huang

Text/document classification

- Sentiment classification

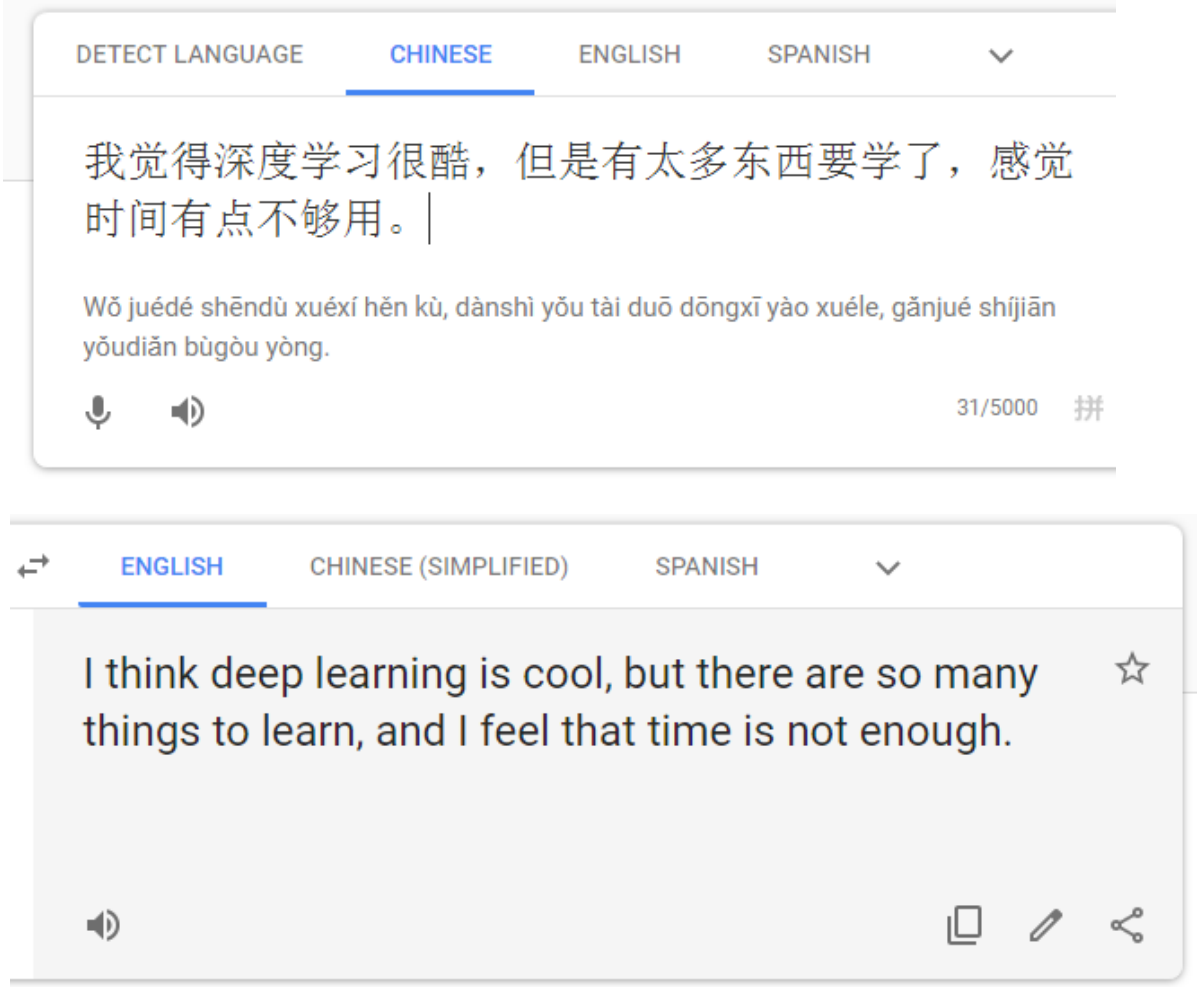
Deep learning is so cool.	Positive
His attitude is annoying.	Negative
I attended the lecture today.	Neutral

- Document classification



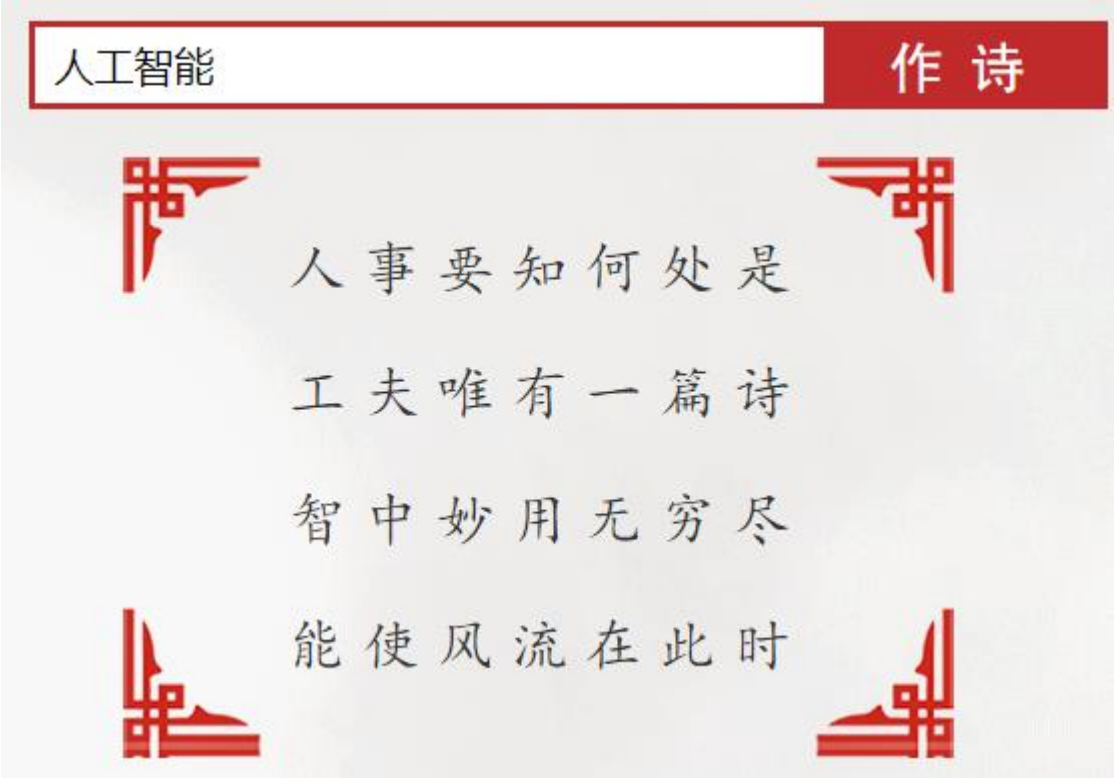
Machine translation

https://translate.google.com

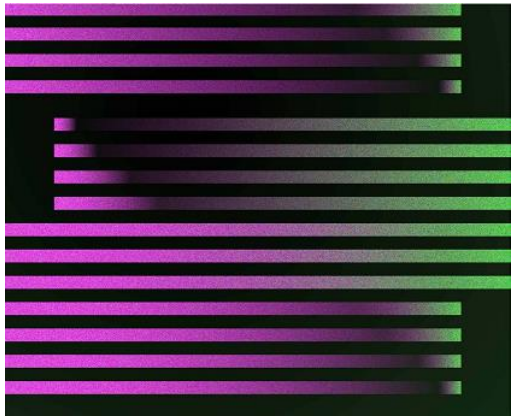


Chinese ancient poetry generation

http://jiuge.thunlp.org/



A recent milestone



ChatGPT

Q: 你觉得在可见的未来现在的金融领域会如何被深度学习影响呢？

A: 深度学习和人工智能技术在金融领域的应用已经越来越广泛，而且未来会继续深入影响金融领域。以下是我认为深度学习可能对金融领域产生影响的几个方面：

- 风险管理和投资决策：深度学习技术能够处理大量结构化和非结构化数据，对金融机构进行风险管理和预测具有重要意义。深度学习算法能够自适应地分析和学习市场中不同的行为和情况，并为投资决策提供更准确和全面的分析和建议。
- 交易：深度学习技术可以分析历史数据和市场趋势，以帮助交易员制定更明智的决策，识别更好的交易机会并执行更快速的交易。诈骗检测：金融行业是一个高风险的领域，因此诈骗检测对金融机构至关重要。深度学习技术可以通过自适应地学习和识别异常模式，从而更准确地检测和预防欺诈。
- 客户服务：深度学习技术可以帮助金融机构更好地理解客户需求，提供更准确的服务和产品推荐，以提高客户体验。

总之，深度学习技术在金融领域的应用将会是一项持续的趋势，它可以帮助金融机构更好地理解市场和客户需求，并提供更准确、更个性化和更高效的服务。

Summary of Part I

Tagging and parsing

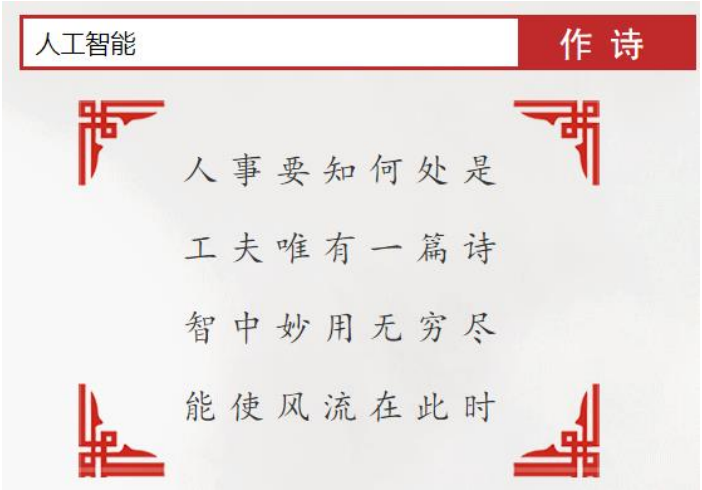
Text generation

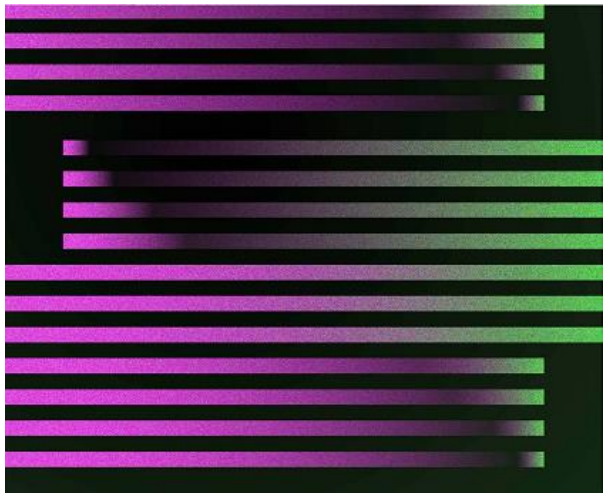
Text/document classification

Sentiment analysis

Machine translation

Question answering





Outline

I. Introduction to NLP
II. Word representation
III. Word2vec
IV. Summary

Word representation

- One-hot representation

  Flower: [0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

  Phone: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0]

  Dimensionality: the vocabulary size $|\boldsymbol{V}|$

- Problems:
  - Dimensionality is high (could be millions)
  - Does not represent the relationship between words

  dist("kid", "child") has nothing to do with dist("flower", "car")

Word representation

- Motivation: You can get a lot of value by representing a word by means of its neighbors

"You shall know a word by the company it keeps"
(J. R. Firth 1957: 11)

…government debt problems turning into banking crises as has happened in…
…saying that Europe needs unified banking regulation to replace the hodgepodge…

These words will represent *banking*

One of the most successful ideas of modern statistical NLP!

Slide from Richard Socher

How to represent a word using its neighbors

use a co-occurrence matrix !

- Two options: full document vs windows
  - Word-document co-occurrence matrix will give general topics (all sports terms will have similar entries) leading to "Latent Semantic Analysis"
  - Window around each word captures both syntactic and semantic information

Slide from Richard Socher

Window based co-occurence matrix

Slide from Richard Socher

- Window length 1 (more common: 5 - 10)
- Symmetric (irrelevant whether left or right context)

Example corpus

I like deep learning.
I like NLP.
I enjoy flying.

counts	I	like	deep	learning	NLP	enjoy	flying
I	0	2	0	0	0	1	0
like	2	0	1	0	1	0	0
deep	0	1	0	1	0	0	0
learning	0	0	1	0	0	0	0
NLP	0	1	0	0	0	0	0
enjoy	1	0	0	0	0	0	1
flying	0	0	0	0	0	1	0

多选题 1分

Setting

What are the problems with this co-occurence matrix representation?

A Dimension is high

B Representation is sparse making models less robust

C Does not represent the relationship between words

D Need to recalculate the representation of all words after adding a word into the vocabulary

Submit

Summary of Part II

One-hot vector

Flower: [0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

Phone: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0]

Co-occurrence matrix

counts	I	like	deep	learning	NLP	enjoy	flying
I	0	2	0	0	0	1	0
like	2	0	1	0	1	0	0
deep	0	1	0	1	0	0	0
learning	0	0	1	0	0	0	0
NLP	0	1	0	0	0	0	0
enjoy	1	0	0	0	0	0	1
flying	0	0	0	0	0	1	0

Outline

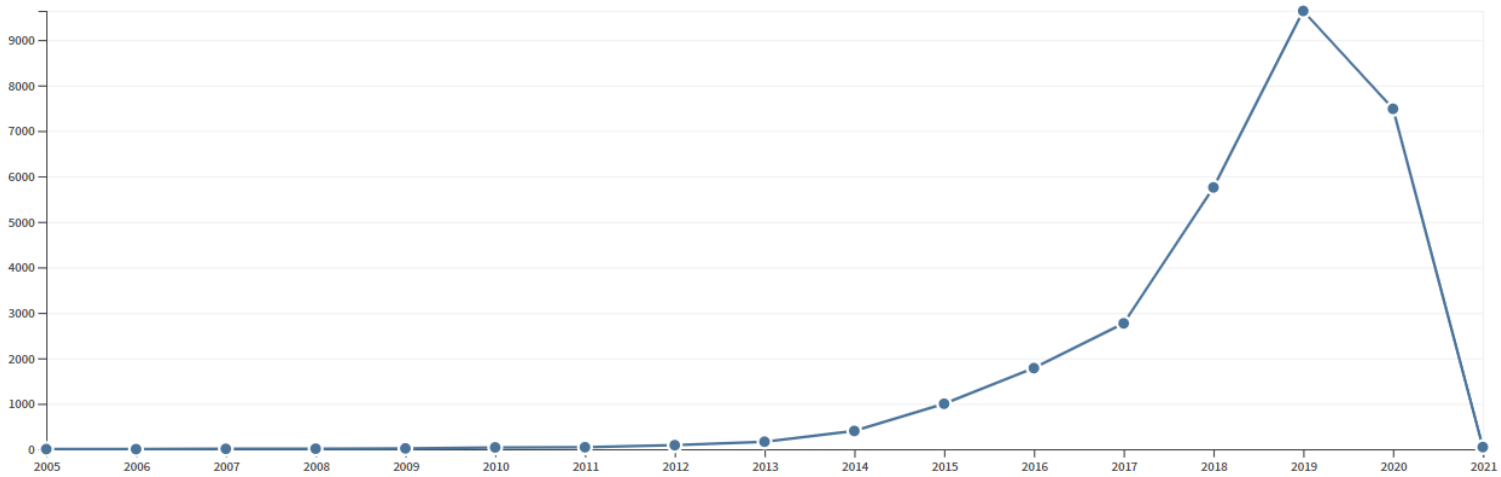
I. Introduction to NLP
II. Word representation
III. Word2vec
IV. Summary

Represent words by low-dimensional vectors

- Idea: store "most" of the important information in a fixed, small number of dimensions: a dense vector
  - Usually around 25 – 1000 dimensions
  - It's easy to perform tasks (classification, generation, etc.) based on this representation
- How to learn the word vectors?
  - A neural probabilistic language model (Bengio et al., JMLR, 2003)
  - A recent simpler and faster model (Mikolov et al., NeurIPS 2013): word2vec → Intro here

Influence of the seminal work

Bengio et al., A neural probabilistic language model. Journal of Machine Learning Research, 2003



From Web of Science



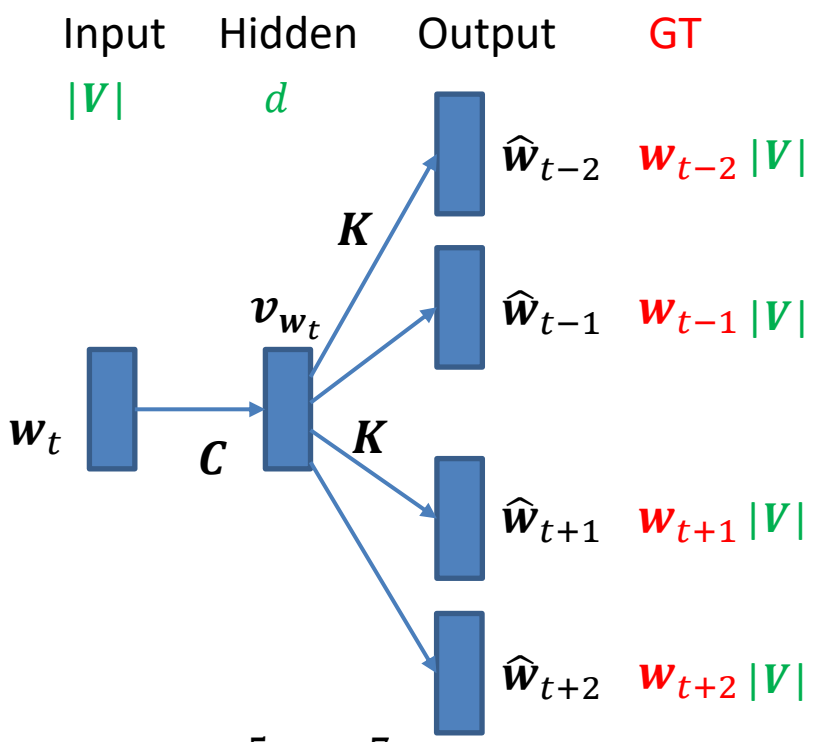
Yoshua Bengio

Turing Award 2018

…In 2000 he made a major contribution to natural language processing with the paper "A Neural Probabilistic Language Model."

A two-layer network

- Instead of capturing co-occurrence counts directly, predict surrounding words of every word

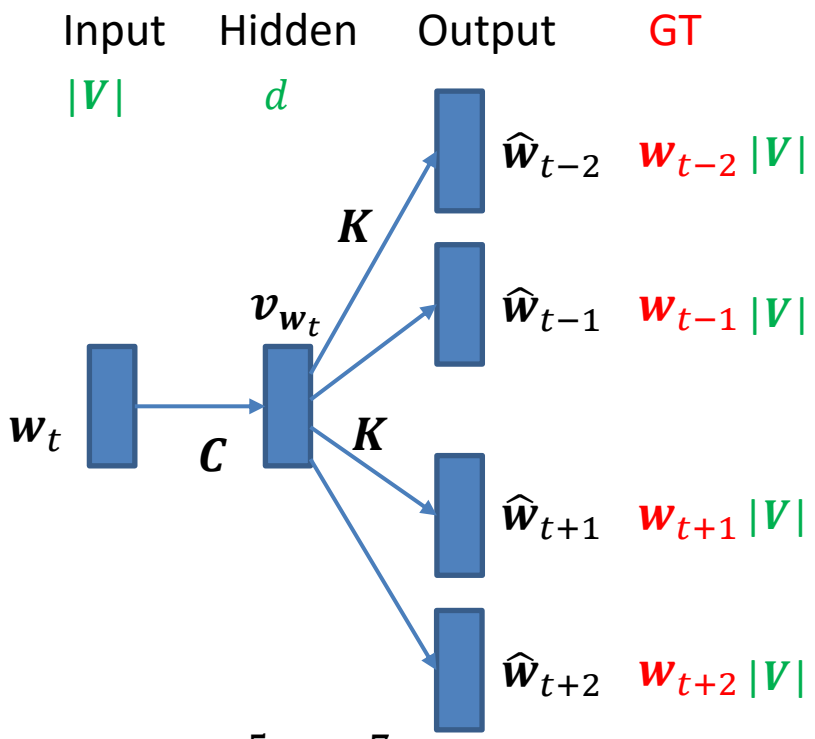


- $w_t \in R^{|V|}, v_{w_t} \in R^d$
- $C \in R^{d \times |V|}$: Each column is the vector of a word in the vocabulary

$|V|$ is about $10^5 \sim 10^7$
d is about $50 \sim 1000$

A two-layer network

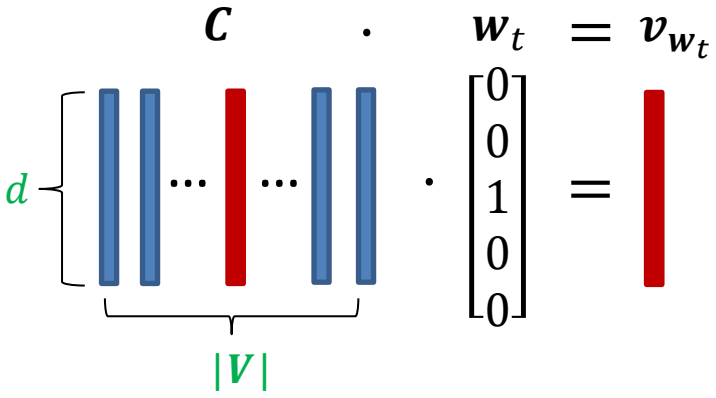
- Instead of capturing co-occurrence counts directly, predict surrounding words of every word



$|V|$ is about $10^5 \sim 10^7$
d is about $50 \sim 1000$

- $w_t \in R^{|V|}, v_{w_t} \in R^d$
- $C \in R^{d \times |V|}$: Each column is the vector of a word in the vocabulary

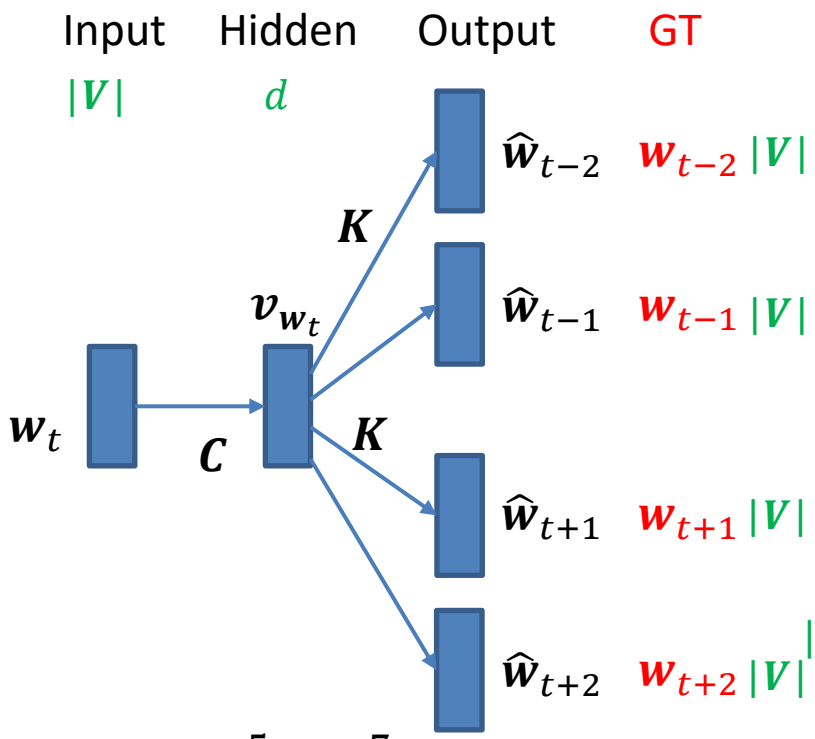
① Project w_t to v_{w_t} via C



The one-hot vector w_t selects the corresponding column in C

A two-layer network

- Instead of capturing co-occurrence counts directly, predict surrounding words of every word

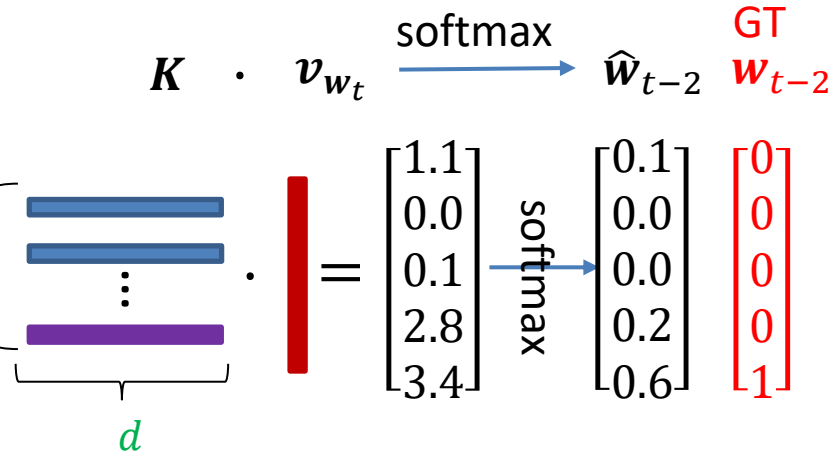


$|V|$ is about $10^5 \sim 10^7$

d is about $50 \sim 1000$

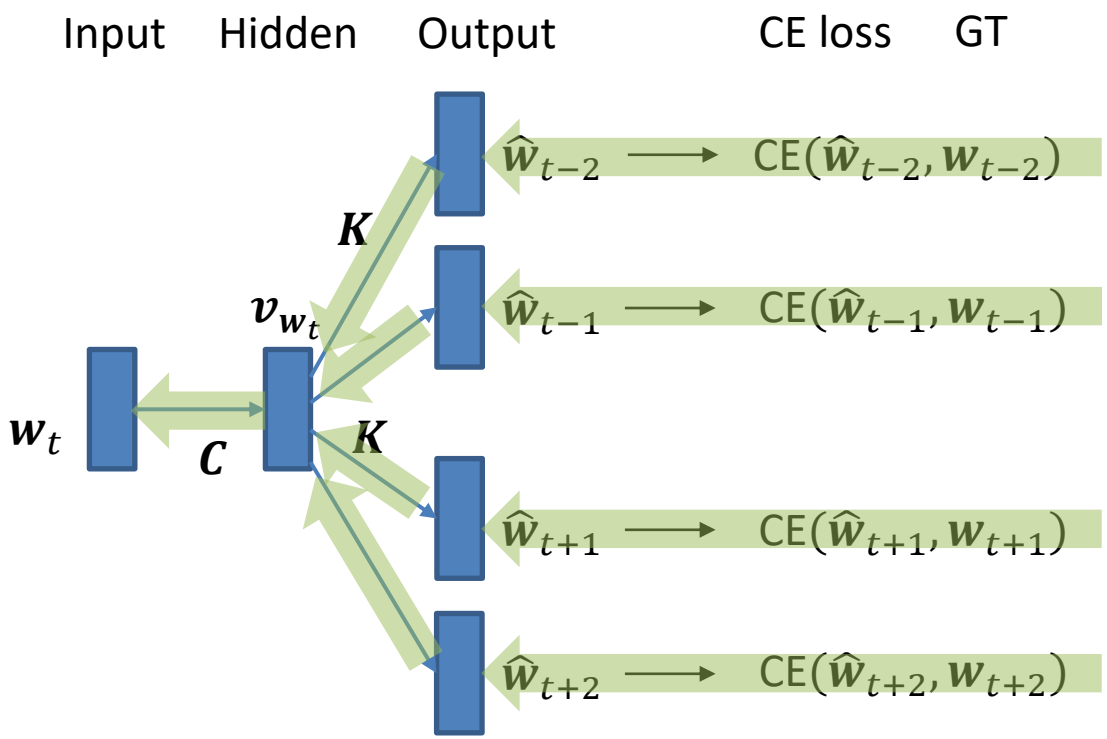
- $w_t \in R^{|V|}, v_{w_t} \in R^d$
- $C \in R^{d \times |V|}$: Each column is the vector of a word in the vocabulary

② Project v_{w_t} to a word at location $t-2$ via $K \in R^{|V| \times d}$



Training

How to training the network?



- The predictions are the same
- The GTs are different

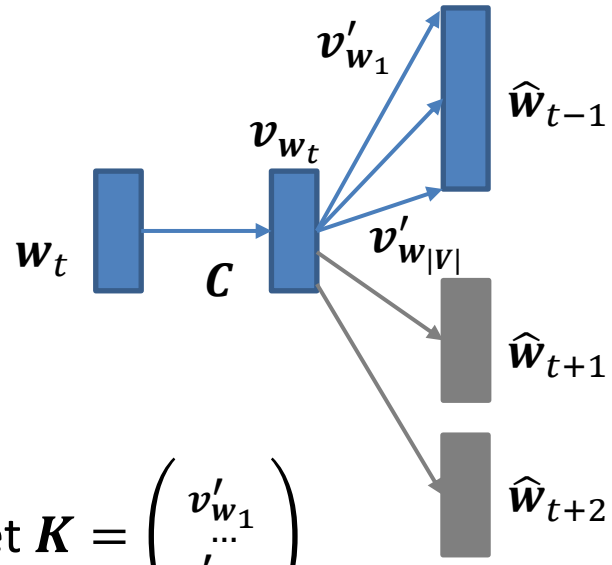
How to distinguish different words in this window?

Training the network with BP

Formulation

- Objective: given an input word, maximize the prob of surrounding words

skip-gram model



Let $\boldsymbol{K} = \begin{pmatrix} v'_{w_1} \\ \cdots \\ v'_{w_{|V|}} \end{pmatrix}$

- The prob of a word $\boldsymbol{w}_k$ at location $t+j$ $(j = -2, -1, 1, 2)$ is

$$p(\boldsymbol{w}_k|\boldsymbol{w}_t) = \frac{\exp\left(\boldsymbol{v}'_{\boldsymbol{w}_k}{}^{\top}\boldsymbol{v}_{\boldsymbol{w}_t}\right)}{\sum_{m=1}^{|V|}\exp\left(\boldsymbol{v}'_{\boldsymbol{w}_m}{}^{\top}\boldsymbol{v}_{\boldsymbol{w}_t}\right)}$$

- During training, a word $\boldsymbol{w}_{t+j}$, is given as the *desired* output word, the cross-entropy loss:

$$-\sum_{k=1}^{|V|} r_{\boldsymbol{w}_k} \ln p(\boldsymbol{w}_k|\boldsymbol{w}_t) = -\ln p\left(\boldsymbol{w}_{t+j}\middle|\boldsymbol{w}_t\right)$$

Objective function

- We have same requirement at other locations in the window. Sum the losses over all locations

$$J^t(\boldsymbol{\theta}) = -\sum_{-c \le j \le c, j \neq 0} \ln p\left(\boldsymbol{w}_{t+j} \middle| \boldsymbol{w}_t\right)$$

  where c is the window size and $\boldsymbol{\theta}$ denote all parameters

- Average over all given input $\boldsymbol{w}_1, \boldsymbol{w}_2, \boldsymbol{w}_3, \ldots, \boldsymbol{w}_T$,

$$J(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T} J^t(\boldsymbol{\theta}) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-c \le j \le c, j \neq 0} \ln p\left(\boldsymbol{w}_{t+j} \middle| \boldsymbol{w}_t\right)$$

- It's equivalent to maximizing the average log probability
- BP algorithm and SGD are used to train the model

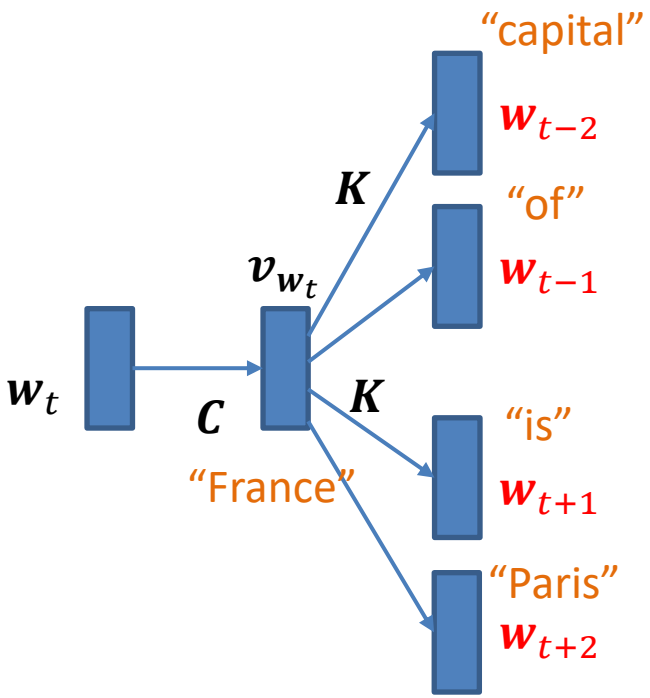
Input vector versus output vector

- Each column of $\boldsymbol{C}$ corresponds to a word: "input vector"
- Each row of $\boldsymbol{K}$ corresponds to a word: "output vector"

- Clearly, we need to predefine the correspondence between the columns of $\boldsymbol{C}$ and the words in the corpus
- In order to do training, the same order of rows must be used in $\boldsymbol{K}$
- After learning, the i-th column of $\boldsymbol{C}$ and the i-th row of $\boldsymbol{K}$ can be averaged to represent the i-th word

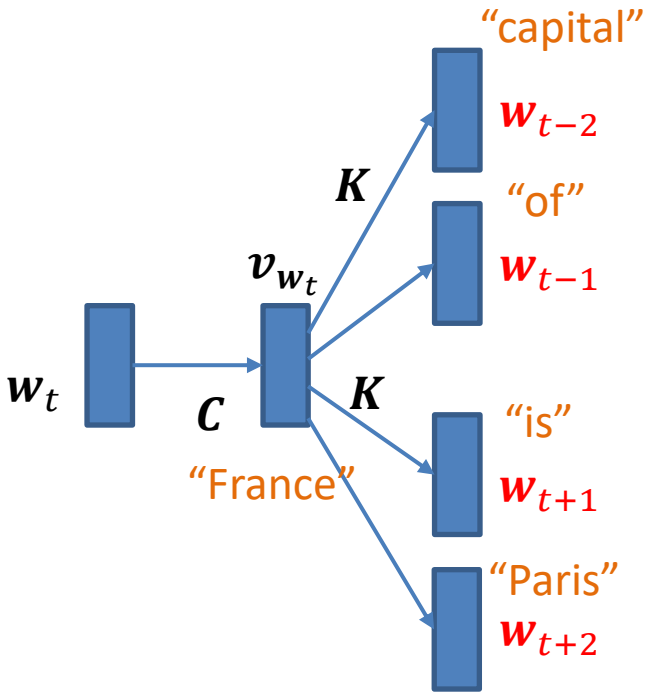
An example

"…capital of France is Paris…"

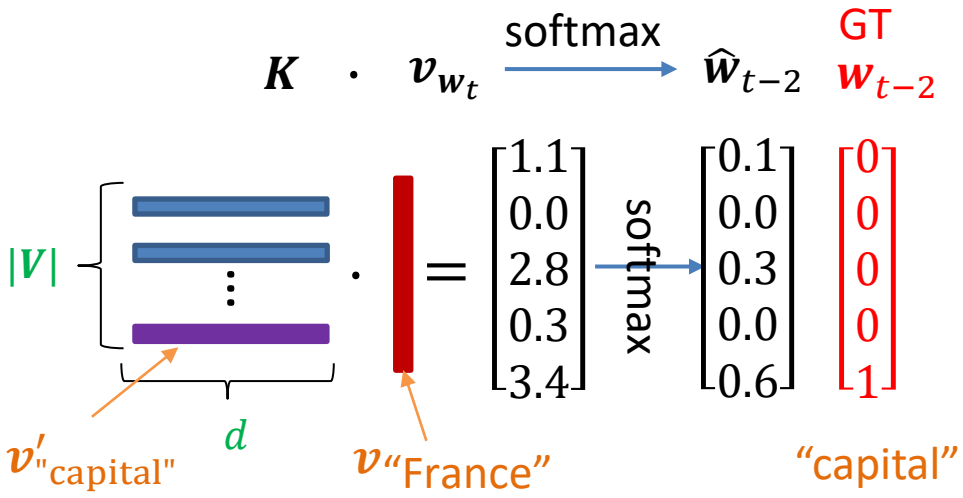


An example

"…capital of France is Paris…"

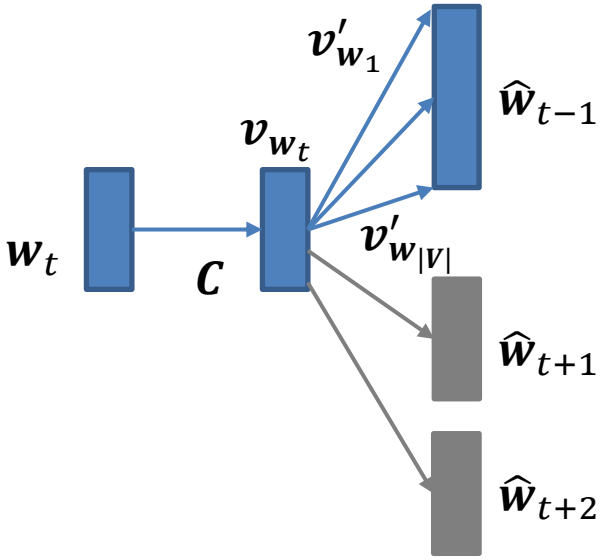


- Project v_{w_t} to a word at location $t-2$



- The row in K corresponding to "capital" should be similar to $v_{\text{"France"}}$
- Similarly, the rows corresponding to "of", "is", "Paris" should also be similar to $v_{\text{"France"}}$

Alternatives to full softmax



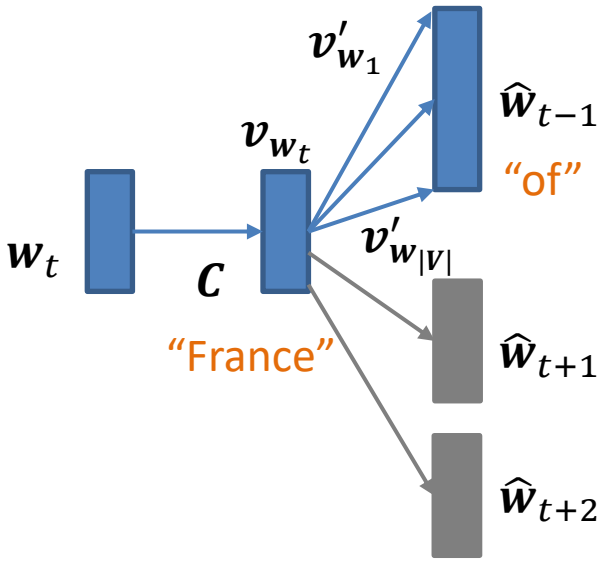
At *each location* in the window

$$p(\boldsymbol{w}_k = \text{"at this location"} | \boldsymbol{w}_t) = \frac{\exp(\boldsymbol{v'_{w_k}}^\top \boldsymbol{v}_{w_t})}{\sum_{m=1}^{|\boldsymbol{V}|} \exp(\boldsymbol{v'_{w_m}}^\top \boldsymbol{v}_{w_t})}$$

where $k = 1, \ldots, |\boldsymbol{V}|$

- Any problem with this formulation?
  - The normalization term is computationally expensive
- Two alternatives
  - Hierarchical softmax (Morin, Bengio, 2005; Mikolov et al., 2013)
  - Negative sampling (Mikolov et al., 2013) -> Introduce here

Motivation



Suppose $t-1$ is in the window

At $t-1$, Softmax calculates the probs:

$p(\text{"of"} = \text{"at } t-1\text{"} | \text{"France"})$

$p(\text{“Beijing”} = \text{“at } t-1\text{”} | \text{“France”})$

$p(\text{“fish”} = \text{“at } t-1\text{”} | \text{“France”})$

…

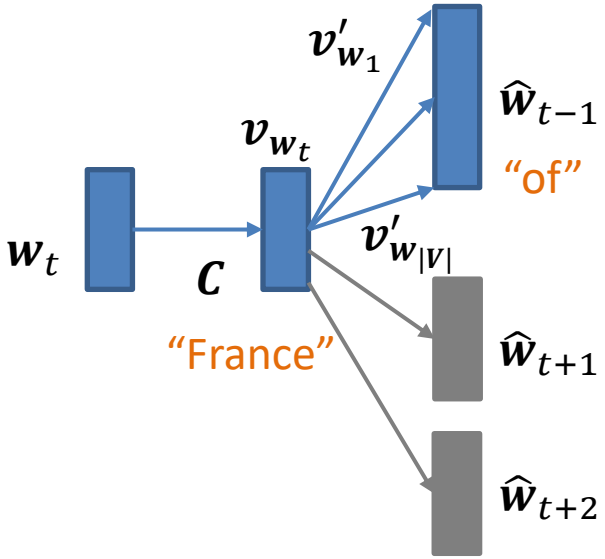
$p(\text{“of”} = \text{“at } t-1\text{”} | \text{“France”})$ is larger than other probs because “of” appears at $t-1$ in the training set

Construct a binary classification problem at $t-1$:

- “of” is at $t-1$
- other $|V|-1$ words are not at $t-1$

Logistic regression!

Motivation



At *each location* in the window, calculate

$$p(\boldsymbol{w}_k = \text{"at this location"} | \boldsymbol{w}_t) = \frac{1}{1 + \exp\left(-\boldsymbol{v}'_{\boldsymbol{w}_k}{}^{\top} \boldsymbol{v}_{\boldsymbol{w}_t}\right)}$$

where $k = 1, \ldots, |\boldsymbol{V}|$

Sigmoid function

Suppose $t-1$ is in the window

At $t-1$, Sigmoid calculates the probs:

$p(\text{"of"} = \text{"at } t-1\text{"} | \text{"France"}) > 0.5$

$p(\text{“Beijing”} = \text{“at } t-1\text{”} | \text{“France”}) < 0.5$

$p(\text{“fish”} = \text{“at } t-1\text{”} | \text{“France”}) < 0.5$

…

$|V| - 1$ terms

Sample K terms, where $K \ll |V| - 1$

Negative sampling

- We use logistic regression to approximate softmax
  - Classify $K+1$ words into two classes: (1) appears in the window; (2) doesn't appear in the window
  - One positive sample, K negative samples from a distribution $P(\boldsymbol{w})$
  - K can be 5~20 for small training datasets, while 2~5 for large datasets

Recall: cross-entropy error for logistic regression (for one input sample)

$$E(\boldsymbol{\theta}) = -t \ln \sigma(\boldsymbol{x}) - (1-t) \ln(1-\sigma(\boldsymbol{x}))$$

$$\ln \sigma(-x)$$

- σ is the logistic sigmoid function
- t is the ground-truth, 1 or 0

Negative sampling

$$p(\boldsymbol{w}_k|\boldsymbol{w}_t) = \sigma\left(\boldsymbol{v}'_{\boldsymbol{w}_k}{}^{\top}\boldsymbol{v}_{\boldsymbol{w}_t}\right) = \frac{1}{1+\exp\left(-\boldsymbol{v}'_{\boldsymbol{w}_k}{}^{\top}\boldsymbol{v}_{\boldsymbol{w}_t}\right)}$$

$k = 1, \dots, K+1$

- The objective function for every word in the window

$$J^c = -\ln\sigma\left(\boldsymbol{v}'_p{}^{\top}\boldsymbol{v}_t\right) - \sum_{i=1}^{K} \mathbb{E}_{\boldsymbol{w}_i \sim P(\boldsymbol{w})} \ln\sigma\left(-\boldsymbol{v}'_i{}^{\top}\boldsymbol{v}_t\right)$$

Max. prob that real words appear in the window

Min. prob that random words appear in the window

  - $\boldsymbol{v}_t$ is the vector of the *input word* $\boldsymbol{w}_t$
  - $\boldsymbol{v}'_p$ and $\boldsymbol{v}'_i$ are the vectors of the positive and negative *output words* $\boldsymbol{w}_p$ and $\boldsymbol{w}_i$, respectively

Overall loss

For softmax (recall):

$$J(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T} J^t(\boldsymbol{\theta}) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\le j\le c, j\neq 0} \ln p\left(\boldsymbol{w}_{t+j}\middle|\boldsymbol{w}_t\right)$$

For negative sampling:

$$J(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T} J^t(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T}\sum_{-c\le j\le c, j\neq 0} J^c$$

where $J^c = -\ln\sigma\left(\boldsymbol{v}'_{\mathrm{O}}{}^{\top}\boldsymbol{v}_t\right) - \sum_{i=1}^{K} \mathbb{E}_{\boldsymbol{w}_i\sim P(\boldsymbol{w})} \ln\sigma\left(-\boldsymbol{v}'_i{}^{\top}\boldsymbol{v}_t\right)$

How to define $P(\boldsymbol{w})$?

Subsampling of frequent words

- In very large corpora, the most frequent words can easily occur hundreds of millions of times (e.g., “in”, “the”, and “a”)
- Such words usually provide less information value than the rare words (e.g., “France”, “lavender”)
- During training,
  - The vectors of frequent words do not change significantly after training on several million examples
  - The vectors of rare words change significantly after training on a small number of examples
- We need to counter the imbalance between the rare and frequent words

Subsampling of frequent words

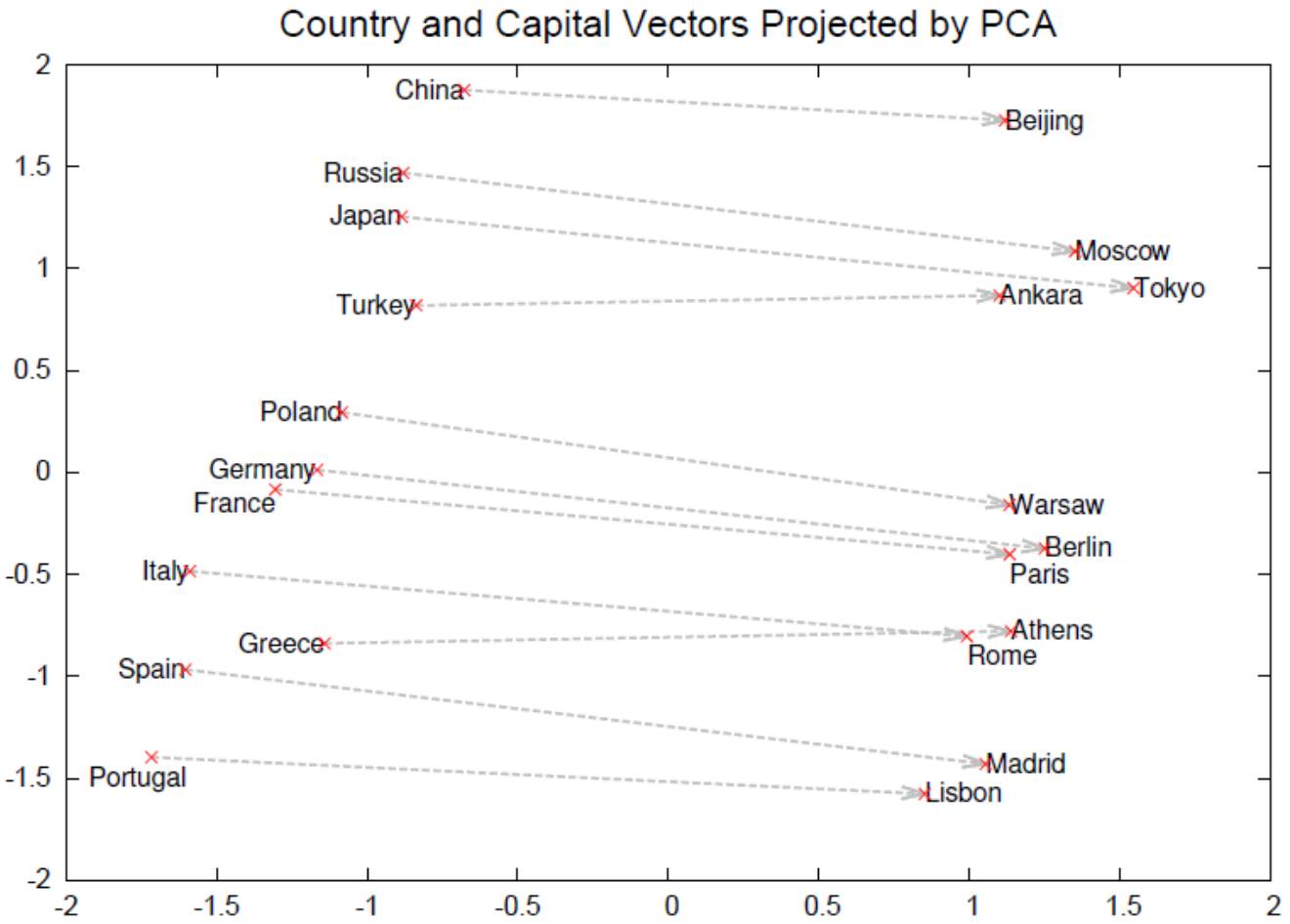
- Each word $\boldsymbol{w}_i$ in the training set is selected with probability (Mikolov et al., 2013)

$$P(\boldsymbol{w}_i) = \sqrt{t/f(\boldsymbol{w}_i)}$$

where $f(\boldsymbol{w}_i)$ is the frequency of $\boldsymbol{w}_i$ in the corpus and t is a threshold, typically around 10^{-5}

Results

Mikolov et al., 2013



2D PCA projection of the 1000-dimensional Skip-gram vectors

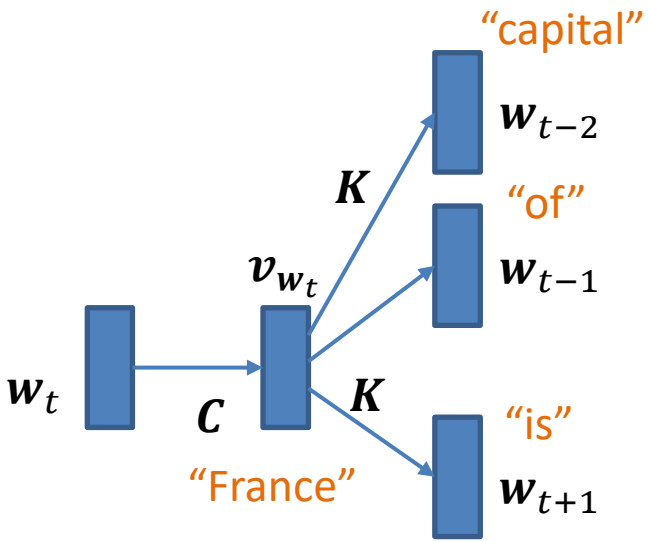
Results

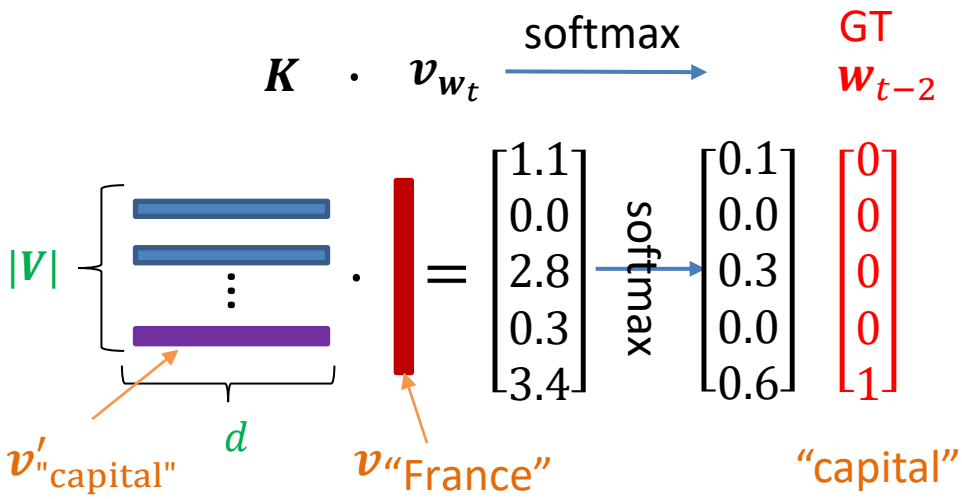
Mikolov et al., 2013

Czech + currency	Vietnam + capital	German + airlines	Russian + river	French + actress
koruna	Hanoi	airline Lufthansa	Moscow	Juliette Binoche
Check crown	Ho Chi Minh City	carrier Lufthansa	Volga River	Vanessa Paradis
Polish zolty	Viet Nam	flag carrier Lufthansa	upriver	Charlotte Gainsbourg
CTK	Vietnamese	Lufthansa	Russia	Cecile De

Table 5: Vector compositionality using element-wise addition. Four closest tokens to the sum of two vectors are shown, using the best Skip-gram model.

Summary of Part III





skip-gram model

Words often appears together would have *similar* vectors!

Negative sampling

单选题 1分

设置

In Negative Sampling, shall we discard frequent words or rare words in the training set?

A Frequent words

B Rare words

提交

单选题 1分

设置

In Word2vec we use a word to predict its surrounding words. Can we use the surrounding words to predict this word to obtain the word embeddings?

A Yes

B No

提交

Outline

I. Introduction to NLP
II. Word representation
III. Word2vec
IV. Summary

Summary of this lecture

Knowledge

I. Introduction to NLP

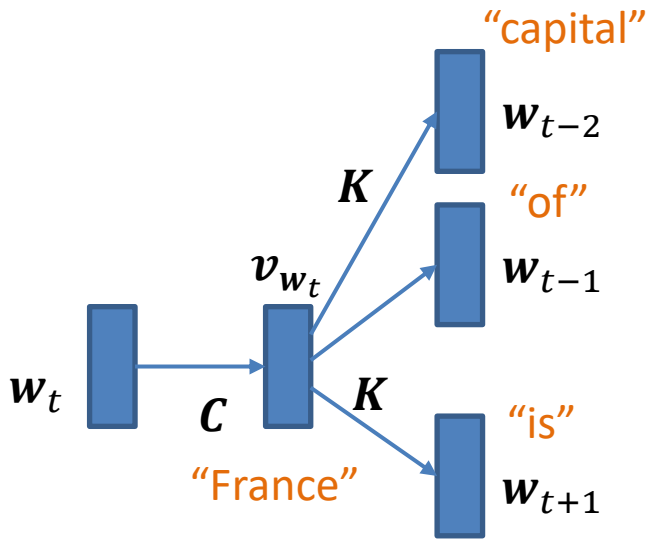
- Tagging and parsing
- Text generation
- Machine translation
- Text/document classification
- Question answering
- Sentiment analysis

II. Word representation

- One-hot vector
- Co-occurrence matrix

counts	I	like	deep	learning	NLP	enjoy	flying
I	0	2	0	0	0	1	0
like	2	0	1	0	1	0	0
deep	0	1	0	1	0	0	0
learning	0	0	1	0	0	0	0
NLP	0	1	0	0	0	0	0
enjoy	1	0	0	0	0	0	1
flying	0	0	0	0	0	1	0

III. Word2vec



Summary of this lecture

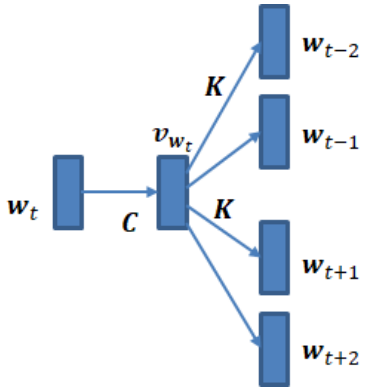
Capability and value

- Represent a word by means of its neighbors

co-occurrence counts

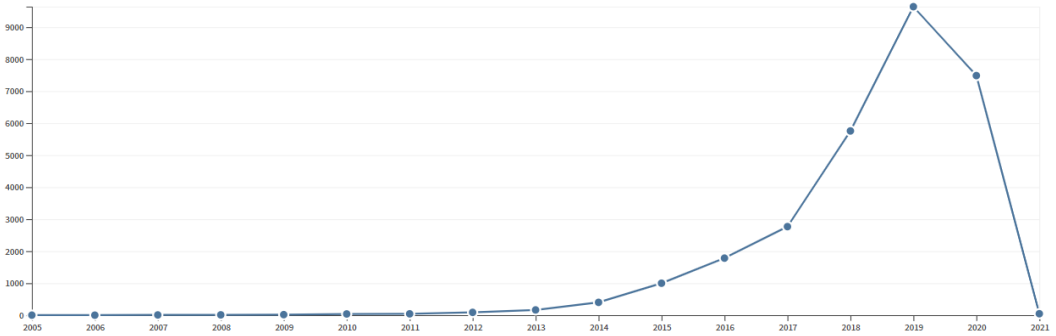
counts	I	like	deep	learning	NLP	enjoy	flying
I	0	2	0	0	0	1	0
like	2	0	1	0	1	0	0
deep	0	1	0	1	0	0	0
learning	0	0	1	0	0	0	0
NLP	0	1	0	0	0	0	0
enjoy	1	0	0	0	0	0	1
flying	0	0	0	0	0	1	0

Predict neighboring words



- Is it the best idea?
- How does the brain do this?
- Do we need word2vec in LLM?

- Good work needs time to be recognized



Recommended reading

- Mikolov, Sutskever, Chen, Corrado, Dean (2013)

  Distributed representations of words and phrases and their compositionality

  NeurIPS

Prepare for the next lecture

- Every course project group prepares a 5-minute presentation with slides for one of the following papers
  - Liang, Hu, "Recurrent convolutional neural network for object recognition," CVPR 2015
  - Bengio et al., "A neural probabilistic language model," Journal of Machine Learning Research, 2003



Yoshua Bengio
Turing Award 2018

…In 2000 he made a major contribution to natural language processing with the paper "A Neural Probabilistic Language Model."